



A Design and Analysis Methodology for Component-Based Real-Time Architectures of Autonomous Systems

Nicolas Gobillot¹ · Charles Lesire² · David Doose²

Received: 2 March 2017 / Accepted: 3 December 2018 / Published online: 19 December 2018
© Springer Nature B.V. 2018

Abstract

The integration of autonomous robots in real applications is a challenge. It needs that the behaviour of these robots is proved to be safe. In this paper, we focus on the real-time software embedded on the robot, and that supports the execution of safe and autonomous behaviours. We propose a methodology that goes from the design of component-based software architectures using a Domain Specific Language, to the analysis of the real-time constraints that arise when considering the safety of software applications. This methodology is supported by a code generation toolchain that ensures that the code eventually executed on the robot is consistent with the analysis performed. This methodology is applied on a ground robot exploring an area. **Categories** (2), (3)

Keywords Robotics software · Real-time analysis · Modelling language

Mathematics Subject Classification (2010) 93C85 · 68Q42 · 68M20

1 Introduction

Nowadays dangerous, repetitive or precision requiring jobs are done by robots like flying drones, industrial assembly arms or medical assistants. In all these cases, human beings can interact with the machines. It is therefore essential to guarantee that every part of the robot software and hardware will produce a safe behaviour. For instance, we need to ensure that these robots will not damage themselves or their environment and more importantly that they do not hurt any human-being. Safety concern has already been considered regarding several aspects of robotics: collision avoidance [1–3], human awareness [4, 5], fault detection [6, 7], or controller synthesis [8, 9].

In this paper we are concerned with the analysis of the fulfilment of real-time constraints on the robot software. Real-time constraints are part of the *non-functional* requirements that arise when guaranteeing the safety of a critical software. Real-time constraints are of course not sufficient, as the *functional* part of the architecture must also be proved. Violating real-time constraints can however lead to inconsistent or unsafe behaviours of the *functional* part of the software architecture. For instance, [10, 11] have analysed the impact of software delays in control systems regarding the system stability. Verifying real-time constraints of embedded control architectures is therefore necessary to guarantee a safe behavior of the robotic system.

1.1 Real-Time Analysis

Real-time guarantees are brought by timing analyses on the software. A real-time analysis is based on the computation of the *Worst Case Response Time* (WCRT) of tasks [12]. The WCRT of a task represents the longest time between its activation and the end of its execution, including all the possible interruptions. During the past decades, the widely known *Liu and Layland* [13] fixed priority models, as well as dynamic priority task models such as *Earliest Deadline First* (EDF, [14]) have been precise enough for software analysis. Such models use a common and simple task model

✉ Nicolas Gobillot
nicolas.gobillot@ifremer.fr

Charles Lesire
charles.lesire@onera.fr

David Doose
david.doose@onera.fr

¹ Ifremer, La Seyne-sur-Mer, France

² ONERA, Toulouse, France

(see Fig. 1): each task τ_i is defined by a period T_i and a deadline D_i . At each period, corresponding to a release date r_k , the task instance has to be executed. The execution, that can possibly be preempted, takes a time C_k (in the figure, $C_k = C_k^{(1)} + C_k^{(2)}$). In classical task model, this computation time is noted C_i and is represented by an upper bound of its value, the *Worst Case Execution Time* (WCET). The response time R_k is then the delay between r_k and the end of execution, and we have to prove that $R_k < D_i, \forall i, k$.

Due to the increasing complexity of the real-time systems, the analysis methods had to be adapted taking into account resource partitioning [15–17]. Other improvements needed to account for task interdependency [18]. All these approaches for real-time analysis of software applications are based on a model of the software. It is then very important to have an execution model which is as close as possible to the practical execution that takes place on robots.

1.2 Software Design in Robotics

Developing autonomous robots has lead researchers to design several concepts to organise functional parts of the software architecture, including, among others, sensing, planning, and acting functions. A classical architecture concept is the 3-level architecture [19], and other paradigms have been proposed in the past decades [20–23]. Such developments are the *application* part of the software architecture: they concern the implementation of the functions embedded on the robot, but they must also settle on a middleware and/or operating system in order to be eventually deployed. Moreover, the development and implementation of these applications encounter some specificity when developing robotic application. They need for fast evolutions that lead to short development cycles of several month unlike in aeronautics or in the nuclear field which have development cycles of tenth of years. Furthermore the complexity of embedded systems software architectures increases with more and more tasks involved, making the use of software design methodologies and tools mandatory. To help the software robot developer, some

recent design approaches used in robotics are based both on a middleware to help the development, and on a component-based approach to help managing the development cycle. Such approaches are typically relevant when the system under study is a critical embedded system (or at least some part of the software is critical). A component-based design pattern allows the software architect to build a robotic architecture by assembling existing software components [24, 25].

Table 1 summarizes several approaches regarding either middlewares for developing robotic applications (top rows) and model-based processes (bottom rows).

Middlewares provide operating system and hardware abstractions. A middleware typically proposes an Application Programming Interface to develop and deploy tasks and threads without taking into account the operating system and thus the hardware specificities. While we can find several robotic-oriented middlewares with real-time capabilities [26, 27, 29, 30], only Orocos-RTT [30] uses a behaviour model of components, based on a Finite State-Machine. Moreover, Orocos-RTT is widely used, in particular because of its integration with ROS [31]. Component-based approaches are often associated with modelling languages, such as Domain Specific Languages (DSL), that allow to develop software components in more abstract languages. Interesting approaches with respect to real-time and behaviour models are summarized in Table 1: BIP [32] and CPAL [33] have their own execution engine; BRIDE [35] and Rock [34] provide model-based design of components and allow to respectively generate ROS and Orocos components; G^{en}oM [36] proposes a rich component behaviour model, and can generate code into several middlewares (e.g., Orocos, ROS). Real-time evaluations in G^{en}oM [37] are only based on raw measurements, and make the hypothesis that functions execution will not be preempted by the operating system. SmartSoft [38, 39] allows to generate real-time code based on the DDS framework and estimates the WCRT using Cheddar [40] without using the component behaviour model.

1.3 Contribution

The literature presented here above shows that there is no toolchain for real-time development of robotic software components that is complete, i.e. that provides a model-based development of real-time components, coupled with an accurate analysis of real-time properties. In this paper, we propose a design and real-time analysis process that is compliant with the current usage of robotic software developers, uses an accurate model of the resulting

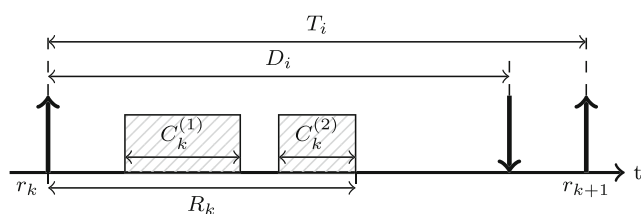


Fig. 1 Task model illustration: k -th execution of i -th task τ_i

Table 1 Summary of related references and their respective features (✓ when supported, ~ when partially supported or limited, ✗ when unsupported)

		component based	real-time	DSL	code generator	behaviour model	WCET	WCRT
Middleware	cisst [26]	✓	✓	✗	✗	✗	✗	✗
	RTC [27]	✓	✓	✗	✓	✗	✗	✗
	RoboComp [28, 29]	✓	✓	✓	✓	✗	✗	✗
	Orocos [30]	✓	✓	✗	✗	✓	✗	✗
	ROS [31]	✓	✗	✗	✗	✗	✗	✗
Model-based	BIP [32]	✓	✓	~	✓	✓	✗	✗
	CPAL [33]	✓	✓	✓	✗	✓	✓	✓
	Rock [34]	✓	✓[30]	~	✓	✓	✗	✗
	BRIDE [35]	✓	✗[31]	✓	✓	✓	✗	✗
	G ^{en} oM [36]	✓	~	✓	✓	✓	~[37]	~[37]
	SmartSoft [38, 39]	✓	✓	✓	✓	✗	✗	~[40]

execution of the application, and provides relevant real-time analysis results that will help developers to improve the overall safety of robotic systems.

Compared to the works presented in Table 1, we have developed a model-based process that generates Orocos code, with accurate WCET/WCRT evaluations based on the component behaviour models. This process (Fig. 2) settles on:

- The design of architecture models using a Domain Specific Language (see Section 3);
- Code generation using state-of-the-art robotic middlewares (see Section 6);
- Architecture execution, along with an accurate execution model (see Section 4);
- A real-time analysis algorithm that computes accurate WCRTs (see Section 5).

Part of this work has already been published:

- a first version of the modelling language has been presented in [41]; here we present an updated version of the language with its complete grammar and concrete code snapshots used in an experiment illustrating the whole process;
- the real-time analysis presented in [42] is further detailed in this paper;

- the application case has been published in [43] with a focus on obtaining WCET from measures; this point is briefly described in this paper, but the application is further detailed, with code snapshots and more complete results.

2 Case Study

The contribution presented in this paper has been applied to several case studies: a ground robot for exploration of indoor environments (Fig. 3a), a ground robot for patrolling missions to secure infrastructures (Fig. 3b), a ground robot for inspection of airport traffic lights (Fig. 3c), and a UAV performing a search-and-track mission [44].

The architectures of the ground robots are very similar regarding the control components. In this paper we use the architecture of the Pioneer P3-DX robot used in the indoor exploration mission, as it provides interesting results that lead to the final discussions of this paper (Section 7.3).

The Pioneer P3-DX robot is made of:

- (1) the mobile base with two motorized wheels to drive the robot and a passive caster wheel,
- (2) seven front-facing ultrasonic range finders and
- (3) an embedded low-level controller to drive the two motors and the ultrasonic sensors.

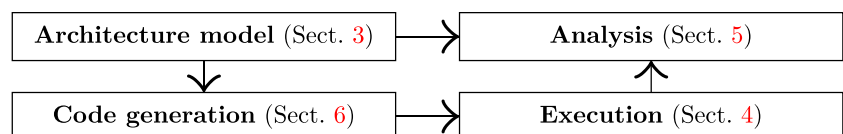
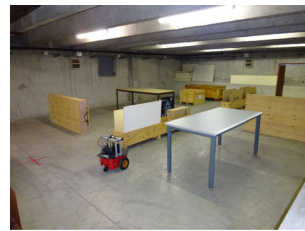
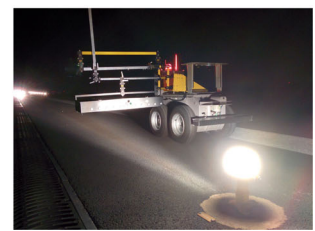
Fig. 2 Sketch of the proposed process

Fig. 3 Case studies

(a) Indoor exploration with a P3-DX robot



(b) Outdoor patrolling with a Summit XL robot



(c) Airport light inspection with a 4MOB robot

On top of this, we added a high-level computer embedding a four core 1.83GHz processor, a laser scanner and a 3D camera. The high-level computer communicates with the proprietary low-level controller through its unique serial 9600bauds RS232 interface.

In the context of our work, we configured the high-level computer to run a real-time patched Linux distribution.

3 The MAUVE Language

As stated in the introduction, we need to catch best practices of robotics software development to build an accurate execution model of the resulting application. This is done by providing a modelling language to the developer. General purposes languages, such as AADL [45], UML [46] or MARTE [47] have been defined to model complex systems and used in several applications such that in the transportation or nuclear industry. These languages may be suited to have an abstract model of a complex robotic system [48, 49], but are limited when we need for an accurate model of execution.

In robotics, and more generally in cyber-physical systems, a lot of works have defined and used DSLs. Contrary to general purpose languages, these DSLs are specific to some domains, then restricting their applicability, but have the good level of abstraction to model very deeply some parts of the system. Some of the works referenced in Table 1 have DSLs to model component-based architectures. However, none of these DSLs allows to model accurately component behaviours and their temporal execution, which are some mandatory elements to perform accurate real-time analyses. We then decided to define a new DSL, MAUVE. A first version of this DSL has been presented in [41]. This section presents an updated version of the model with better notations, particularly for the data ports access and parameter accessors for the codels. Moreover, we provide here the complete grammar of the

MAUVE language, as it is a good way to highlight the several concepts of the language. However, note that the grammar itself is not sufficient to describe all the MAUVE model, as it also settles on complementary tools such that a type checker (to check the type of ports, components, ...) and some semantic analysis rules (like avoiding multiple definitions of the same element). Moreover, to present MAUVE in a concise way, we have omitted some language primitives that could be trivially defined (like strings definition for type names and element ids).

3.1 Codels Declaration

Codels are declarations of functions that may be used by the components (see Listing 1). Only the function prototype is defined at the model level, the function core being implemented in a target language. The term *codel*, as originally proposed by [36], stands for elementary code.

Listing 2 declares the codel for the A* algorithm [50]: the `astar` function takes as argument a start pose `s`, a goal pose `g`, a map, and the resolution of the planning graph (argument `planner_resolution`). It returns the path between `s` and `g`.

3.2 Component Specification

A component [51] is a unit of composition with contractually specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to composition. Therefore, in order to help composition and modularity, we decompose the specification of a component into a *shell* and a *core*.

3.2.1 Component's Shell

The shell of a component defines its interface, i.e. its inputs and outputs. We propose four types of interfaces (see Listing 3):

Listing 1 Part of the MAUVE grammar defining codels

```

1 <codel> ::= 'codel' <id> '(' <arg-list>? ')' ':' <type>
2 <arg-list> ::= <arg-access>? <id> ':' <type>
3           | <arg-access>? <id> ':' <type> ',' <arg-list>
4 <arg-access> ::= 'in' | 'out' | 'inout'

```

Listing 2 Declaration of a codel for the A* algorithm

```
1 codel astar(in s: PoseStamped, in g: PoseStamped, in map:
  OccupancyGrid, in planner_resolution: double): Path
```

- *constants*, which are static typed values associated to a shell. Constants are defined and initialized at the shell's specification.
- *properties*, which are typed component parameters, generally set at instantiation or deployment time. Properties may also have a default value.
- *data ports*, similar to the *push* pattern proposed by [52], are used to publish data from/to a component; they are typed and oriented.
- *operations*, similar to the *query* pattern proposed by [52], are used to call functions or send requests to components; operations can be *provided* or *required*.

Constants and properties can only be modified from the architecture view, whereas ports and operations are used for component to component interactions. Values of properties, as well as connections of ports and operations are not initialized at the moment of specifying a component shell. Instead, they are defined when instantiating and connecting components, at the *architecture* specification step. Type and parameter lists are defined similarly to argument list of Listing 1. Property expressions (not described here by lack of place) allow to write simple arithmetic instructions on integers, floats and strings.

Listing 4 shows the shell specification of the *Navigation* component. The role of this component is on one hand to compute a path from a starting position to a goal on a map, and on the other hand to manage the execution of this path by iteratively sending the path points one after the other.

Its properties are: *resolution*, that specifies the resolution of the map used for path planning (in meters per cell, values between 0.1 and 1, default to 0.3), and *position_threshold*, the threshold used to consider that the current point has been reached (in meters, values between 0 and 1, default to 0.3). The component also has three input ports *map* (the current map known by the robot), *pose* (the current pose of the robot) and *goal* (the goal pose) and the output port *next_point* (on which the path points are published iteratively).

3.2.2 Component's Core

The core of a component defines its behaviour and has to be associated with a shell. It is defined by a state-machine. For specific cases where the state-machine only contains one state, the developer can just define the *update* function instead of the *statemachine* element.

The core of a component is made of several elements (see Listing 5):

- *variables*: internal elements storing data used in the MAUVE expression language;
- *provided operations*: have been declared in the shell and must be defined here;
- *handlers*: allowing to call required operations;
- *programs*: five different programs can be defined to execute some computations:
 - *configure*: this program is called to initialize the component;
 - *start*: this program is used to start the component execution. It can only be called if the *configure* step has executed successfully;
 - *update/statemachine*: this program is executed when the component is running. It can only be called if the component has been started;
 - *stop*: this program is used to stop the execution of the component;
 - *cleanup*: this program is called to clean the component.

Programs are defined using a tiny expression language that allows to evaluate conditions (*if/then/else*), assign values to variables, get status of port (*no_data*, *new_data*, *old_data*), get status (*failure*, *success*, *not_ready*) of handlers (remotely called required operations), call codels, read and write data on ports, call remote operations (*send*, *collect*), and use classical structures (blocks, parentheses).

Listing 3 Part of the MAUVE grammar defining shells

```
1 <shell> ::= 'shell' <id> ('extends' <type-list>)? '{' <shell-ft>* '}',
2 <shell-ft> ::=
3   'constant' <id> ':' <type> (<interval>)? ('=' <prop-expr>)
4   | 'property' <id> ':' <type> (<interval>)? ('=' <prop-expr>)
5   | ('input' | 'output') 'port' <id> ':' <type>
6   | ('require' | 'provide') <id> '(' <param-list>? ')' ':' <
   type>
7 <interval> ::= '[' <prop-expr> ';' <prop-expr> ']'
```


Listing 4 Shell of the *Navigation* component

```

1 shell NavigationShell {
2   property resolution: double [0.1;1] = 0.3
3   property position_threshold: double [0;1] = 0.3
4   input port map: OccupancyGrid
5   input port pose: PoseStamped
6   input port goal: PoseStamped
7   output port next: PoseStamped
8 }

```

Listing 6 shows the specification of the *P3DX* component's core and programs. The connection with the robot is initialized in *configure*. Data exchange with the robot is started in *start*. In *update*, data from the robot are published to output ports, and in case of a new command received in the input port *command*, it is sent to the robot. The robot is stopped (velocity set to 0) in *stop*. Finally, the connection is shutdown in *cleanup*.

3.2.3 State Machines

The behaviour of each component is defined by a state machine. In case of a trivial one-state state-machine, the developer can directly define the update program as shown in Listing 6. A state-machine consists of a set of states linked by a set of transitions. MAUVE state-machine structure (see Listing 7) is very close to the UML/StateChart definition, also used in the Orocos middleware [53].

Each state s_i contains up to four methods: $entry_i$, run_i , $handle_i$ and $exit_i$. The $entry_i$ method contains code executed whenever the state-machine enters state s_i . The run_i method contains the core of the state, executed each time the state-machine is in state s_i . The $handle_i$ method is executed each time the state-machine stays in state s_i , after run_i has been executed. The $exit_i$ method is executed when leaving state s_i . Transitions are guarded and may contain a program executed when triggered.

Listing 8 describes the specification of state Navigating of the *Navigation* component. In this state, the component executes the computed path by sending successive goals.

3.3 Architecture Model

The architecture language (see Listing 9) allows to import an existing (partial) architecture, instantiate components, connect output ports to input ports, possibly defining the size of the buffer when the connection uses a circular buffer,

connect provided operations to required operations, and set property and constant initial values. Listing 10 shows the specification the architecture drawn in Fig. 4.

This architecture is made of three main components [54] designed to compute a path to follow (*navigation*), to avoid obstacles (*guidance*), and to control the robot speed (*control*). Other components provide sensor information (*hokuyo*), robot's status (*p3dx_driver*), or commands from a human operator (*teleop*). Moreover, we have connected a Simultaneous Localization And Mapping (SLAM) component (*gmapping*) to build a map of the environment. The *pose* component fuses information from SLAM and odometry to provide an accurate pose estimate. The *switch* component allows to select manual and automatic speed control. Finally, the *exploration* component selects unexplored areas on the map and send a new goal to navigate to.

4 Software Architecture Deployment

The MAUVE language has allowed to model and develop on one hand the basic components of our system, and on the other hand, architectures that assemble components to build applications. The next step is to deploy and execute these architectures on the final target. In this section, we present first the execution model of the components, and we then present how we can define the execution model using the MAUVE DSL *deployment* specification. As the task model will also need some information about the WCETs, we present some methods used to estimate WCET.

4.1 Task Model

All the components of the software architecture are mapped onto operating system tasks. The components are designed to be temporally independent, and we then allocate each component to a specific task.

Listing 5 Part of the MAUVE grammar defining cores

```

1 <core> ::= 'core' <id> '(' ('<shell-t>' | '<c-ft>*' <c-bh>*' | '<handler>' <id> ':' <type> ('=' <expr>)? (';' )?
2 <c-ft> ::= 'var' <id> ':' <type> ('=' <expr>)? (';' )?
3 | 'handler' <id> ':' <type>
4 | 'provide' <id> '(' ('<param-list>?' | '<program>' | '<cleanup>' '=' <program>
5 <c-bh> ::= 'configure' '=' <program> | 'start' '=' <program> | 'stop' '=' <program>
6 | 'update' '=' <program> | <statemachine>
7

```

Listing 6 Core of the *P3DX* driver component

```

1 core P3DXCore (P3DXShell) {
2   var cmd: TwistStamped; var vel: TwistStamped; var p: PoseStamped;
3   var robot: ArRobotPtr; var connector: ArRobotConnectorPtr;
4   configure = {
5     aria_init(robot, connector, device, baudrate);
6     return aria_connect(robot, connector);
7   }
8   start = { return aria_start(robot, sonarOn, motorOn); }
9   update = {
10    if (read(command, cmd) == new_data) then {
11      aria_command(robot, cmd);
12    } else {
13      aria_loop(robot);
14      p = aria_pose(robot, odometry_frame);
15      write(pose, p);
16      vel = aria_velocity(robot, robot_frame);
17      write(velocity, vel);
18    }
19    stop = { aria_stop(robot, true, true); }
20    cleanup = { aria_disconnect(robot); }
21 }

```

Listing 7 Part of the MAUVE grammar for state machines

```

1 <statemachine> ::= 'statemachine' '{' <sm-feature>* '}'
2 <sm-feature> ::= 'var' <id> ':' <type> ('=' <expr>)? ';'
3 | <state>
4 <state> ::= ('initial')? 'state' <id> '{' <st-method>* <trans>* '}'
5 <st-method> ::= 'entry' '=' <program> | 'exit' '=' <program>
6 | 'run' '=' <program> | 'handle' '=' <program>
7 <trans> ::= 'transition' <id> ('if' <expr>)? 'to' <id> (<program>)?

```

Listing 8 State Navigating of the *Navigation* component

```

1 state Navigating {
2   entry = {
3     current_goal = getPathElement(path_, next_index);
4     write(next, current_goal);
5   }
6   run = {
7     new_received = (read(goal, g) == new_data);
8     read(pose, p);
9     prev_index = next_index;
10  }
11  exit = { next_index = next_index + 1; }
12  transition new_goal_received if (new_received) to Planning
13  transition path_done if ( (dist(p, current_goal) <
14    position_threshold) && (next_index == last_index)) to
15    Arrived
16  transition next_waypoint if (dist(p, current_goal) <
17    position_threshold) to Navigating
18 }

```

Listing 9 Part of the MAUVE grammar for architectures

```

1 <architecture> ::= 'architecture' <id> '{' <archi-feature>* '}'
2 <archi-feature> ::= 'import' <architecture>
3 | 'instance' <id> ':' <type>
4 | 'connection' <id> '.' <id> '->' ('[' <integer> ']')?
5   <id> '.' <id>
6 | 'operation' <id> '.' <id> '->' <id> '.' <id>
7 | 'property' <id> '.' <id> '=' <prop-expr>
8 | 'constant' <id> '.' <id> '=' <prop-expr>
9 <instance> ::= <id> '.' <id>

```

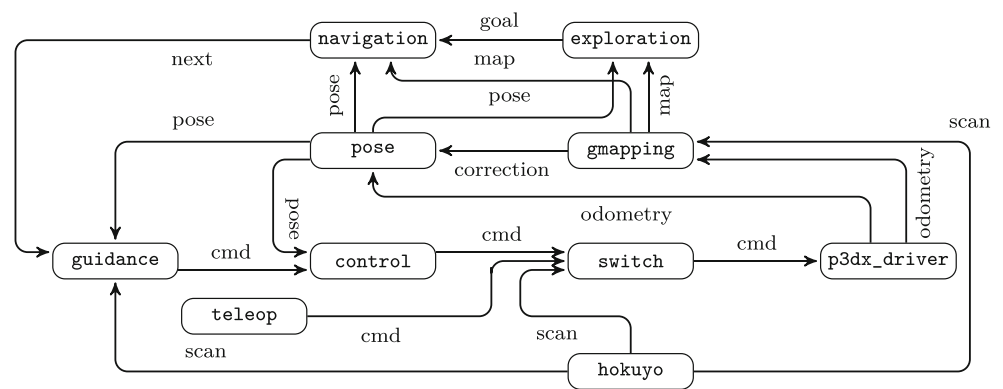
Listing 10 Exploration architecture. The architecture imports an existing navigation architecture and extends it with the exploration component and the hokuyo and P3DX driver components

```

1 architecture ExplorationP3dxArchitecture {
2   import NavigationArchitecture
3   instance exploration: Exploration
4   instance hokuyo: Hokuyo
5   instance p3dx_driver: P3DX
6   connection pose.target -> exploration.pose
7   connection exploration.goal -> navigation.goal
8   connection gmapping.map -> exploration.map
9   connection p3dx_driver.pose -> gmapping.odometry
10  connection p3dx_driver.pose -> pose.source
11  connection hokuyo.scan -> switch.scan
12  connection hokuyo.scan -> guidance.scan
13  connection hokuyo.scan -> gmapping.scan
14  connection switch.command -> p3dx_driver.command
15 }

```

Fig. 4 The component-based exploration architecture



Property 1 When deploying a component, it is associated with a unique task.

Moreover the components exchange data through either asynchronous data ports and operation calls. These protocols allow a component to receive a data from another component without waiting the data to be ready.

Property 2 Data exchange between components are non-blocking.

These properties are ensured in the implementation by the Orocos middleware. Orocos first allows to execute components in threads scheduled by the OS real-time scheduler. Orocos moreover implements data sharing between components using a lock-free mechanism [55] that guarantees non blocking read/write in bounded time, and is not subject to priority inversions.

Definition 1 A component $comp_i$ is executed by a task τ_i defined by four elements:

- the *period* T_i defines the execution frequency;
- the *deadline* D_i sets the moment when the task has to end its execution;
- the *priority* P_i is managed by the scheduler to decide which task to execute;
- the *behaviour* SM_i is defined by the internal structure of the task state-machine.

The following hypothesis is assumed true for the rest of the development.

Hypothesis 1 Deadlines are lesser or equal to the periods:
 $\forall i, D_i \leq T_i$ (1)

The temporal behaviour of a task depends on the activities of the tasks and on the state-machine's behaviour.

Definition 2 A state-machine SM is a tuple $(\mathcal{P}, S, E, entry, exit, handle, run, guard, exec, \delta)$ where:

- $\mathcal{P}$ is a set of *programs*; S is a set of *states*;
- $E \subset S^2$ is a set of *transitions* where each transition e is a pair $(src(e), tgt(e))$;
- $entry : S \rightarrow \mathcal{P}$ associates an *entry* program to a state;
- $exit : S \rightarrow \mathcal{P}$ associates an *exit* program to a state;
- $run : S \rightarrow \mathcal{P}$ associates a *run* program to a state;
- $handle : S \rightarrow \mathcal{P}$ associates an *handle* program to a state;
- $guard : E \rightarrow \{\top, \perp\}$ associates a *guard* expression to a transition; $guard(e)$ evaluates to true ($\top$) when the transition is enabled, and false ($\perp$) when the transition is disabled;
- $exec : E \rightarrow \mathcal{P}$ associates a program to a transition;
- $\delta : \mathcal{P} \rightarrow \mathbb{R}$ associates an execution time to each program.

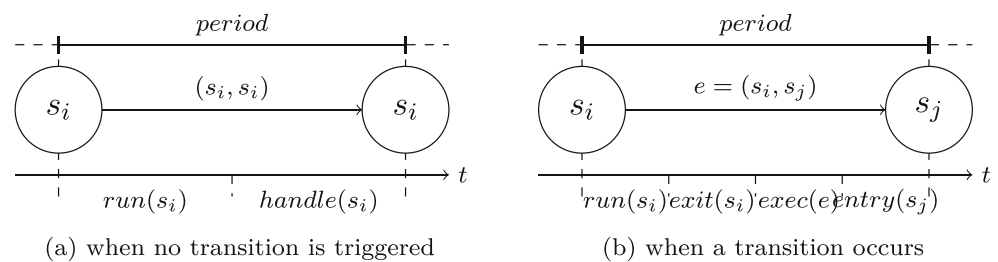
At each period of execution, two execution sequences are possible depending on the transitions that are enabled. Let's assume that current state is s_i , then:

- if no transition is triggered (2), then the component stays in state s_i , and executes $run(s_i)$ and $handle(s_i)$ (Fig. 5a);

$$\forall e \in E, (src(e) = s_i) \Rightarrow (guard(e) = \perp) \quad (2)$$

- if a transition e is triggered (3) then the component goes from s_i to $s_j = tgt(e)$, and executes $run(s_i)$, $exit(s_i)$, $exec(e)$ and $entry(s_j)$ (Fig. 5b).

$$(src(e) = s_i) \wedge (guard(e) = \top) \quad (3)$$

Fig. 5 Execution model of state machines

4.2 Deployment Specification

MAUVE allows to specify a *deployment* by defining the activities of each component compliant with Definition 1. The developer can define the priority, period and deadline of each component, and allocate it to a specific CPU with the affinity (Listing 11).

4.3 Estimating Worst-Case Execution Times

The task model presented in Definition 2 needs the timing function δ in order to perform the real-time analysis presented in Section 5. However, this execution time is generally not deterministic, and the WCET must then be estimated [56]. Static analysis [57] takes the control flow graph and a model of the hardware to compute the worst number of instructions. Static analysis methods have some limitations: (1) they need a model of each instruction, which is inappropriate when using IO functions for instance and (2) they need an accurate model of the hardware, meaning that the approach efficiency is processor-dependant.

On the contrary, Measurement-Based Probabilistic Timing Analysis (MBPTA) [58, 59] uses measurements of execution times, and infer the probabilistic WCET (pWCET) estimation using Extreme Value Theory (EVT). Application of these methods is discussed in Section 7.

5 Real-Time Analysis

To ensure the schedulability of the task system described by the MAUVE DSL, we provide a real-time analysis

process checking if the component's WCRTs are lower than their deadlines. The real-time analysis problem has been tackled many times using more and more complex approaches. The first ones were based on the *Liu and Layland* model [13], using fixed priority preemptive tasks, later extended to dynamic priority tasks such as EDF [14]. In order to cope with increasing application complexity, the task models are more and more refined: the Multiframe model [60, 61], the Digraph Real-Time model [62] or the Digraph Synchronous Finite State-Machine model [63] further increase the complexity and the precision of the analyses using graph based algorithms to compute WCRTs. In this section, we describe a computationally efficient and precise WCRT algorithm that is applicable to our specific task model.

The proposed process (see Fig. 6) needs the WCETs of the components and of their internal codels (see Section 4.3). From these components we extract *Periodic State-Machines* (PSM) which represents the component's timed behaviour (Section 5.1). The WCRTs are then computed from the PSMs in three steps: the first one computes *traces* from the PSMs, a trace being a timeline of one possible PSM execution. These traces are used to deduce an upper bound of all the possible PSM paths, then these upper bounds allow the computation of the WCRTs (Section 5.2).

5.1 Periodic State-Machine

In order to analyse the components behaviour, we model them as Periodic State-Machines (PSM), with the same temporal behaviour as the task it models.

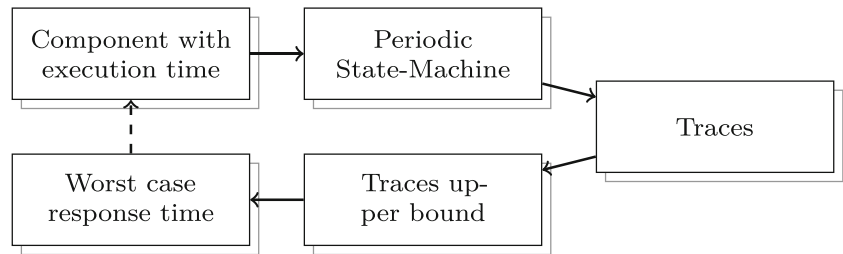
Listing 11 Part of the MAUVE grammar for deployments

```

1 <deployment> ::= 'deployment' <id> '{' 'architecture' <id> <df>* '}'
2 <df> ::= <activity>
3 | ('property' | 'constant') <id> '.' <id> '=' <prop-exp>
4 <activity> ::= 'activity' '{' ('affinity' '=' <integer>)?
5 'priority' '=' (<integer> | 'background')
6 'period' '=' <integer> ('deadline' '=' <integer>)? '}'

```

Fig. 6 Real-time analysis process



Definition 3 Each component is associated with a Periodic State-Machine PSM defined by as a set of states S (similar to the states of the state-machine SM , see Definition 1) and a set of transitions Σ such that:

$$\Sigma = E \cup \{(s, s) \mid s \in S\} \quad (4)$$

The set of transitions Σ contains all the original transitions of the state-machine (E) plus all the loops over states of S .

Property 3 *The Periodic State-Machine fires a transition at every execution period.*

Property 4 *A Periodic State-Machine is strongly connected: every state can be reached from any state through a sequence of transitions.*

$$\forall s_i, s_j \in S, \exists \sigma_1 \dots \sigma_n \in \Sigma \mid s_i \xrightarrow{\sigma_1} \dots \xrightarrow{\sigma_n} s_j \quad (5)$$

This assumption is useful, because it avoids considering that some dead states exist in the task behaviour, and it is a needed assumption for the WCRT computation (see Section 5.2). Moreover, this assumption is a good practice that should be enforced when developing components: it allows to be able to put the task again in its initial state at execution when a faulty behaviour occurs.

Definition 4 A timing function δ is defined over the PSM transitions, such that:

$$\forall \sigma \in \Sigma, \sigma = (s_i, s_j), \delta(\sigma) = \begin{cases} \delta(\text{run}(s_i)) + \delta(\text{exit}(s_i)) + \\ \delta(\text{entry}(s_j)) + \delta(\text{exec}(\sigma)) & \text{if } s_i \neq s_j \\ \delta(\text{run}(s_i)) + \delta(\text{handle}(s_i)) & \text{if } s_i = s_j \end{cases} \quad (6)$$

where the execution time of each program is defined in Definition 2.

5.2 WCRT Computation

In order to compute the WCRTs (i.e., the worst time between task release and the end of its execution), we first use the PSMs structure to extract *traces* representing all the possible execution sequences of the state-machines.

5.2.1 Traces

We define a trace $\mathcal{T}$ as an ordered sequence of transitions:

$$\mathcal{T} = \langle \sigma_1, \dots, \sigma_N \rangle \in \Sigma^N \quad (7)$$

We also define operators to access the i^{th} transition of a trace, and the number of transitions:

$$\forall i \in \llbracket 1..N \rrbracket, \mathcal{T}[i] = \langle \sigma_1, \dots, \sigma_i, \dots, \sigma_N \rangle[i] = \sigma_i \quad (8)$$

$$|\mathcal{T}| = |\langle \sigma_1, \dots, \sigma_N \rangle| = N \quad (9)$$

The *request bound function* (rbf , [64]) is the maximum processor request over a time interval. The tasks are executed periodically then the rbf is a piecewise-constant and increasing function. The rbf of a trace $\mathcal{T}$ at time t is noted $rbf(\mathcal{T}, t)$ and is defined as:

$$rbf(\mathcal{T}, t) = \sum_{i=0}^{\lfloor \frac{t}{T} \rfloor} \delta(\mathcal{T}[i+1]) \quad (10)$$

with $\delta(\sigma)$ the time taken by transition σ (see Definition 4). We then define the relation $\leq$ on traces by comparing the rbf of the traces:

$$\mathcal{T} \leq \mathcal{T}' \Leftrightarrow \forall t, rbf(\mathcal{T}, t) \leq rbf(\mathcal{T}', t) \quad (11)$$

The *next* operator represents the set of traces that follows a given trace:

$$\text{next}(\mathcal{T}) = \{ \langle \mathcal{T}, \sigma \rangle \mid \sigma \in \Sigma \wedge \text{tgt}(\mathcal{T}[\lceil |\mathcal{T}| \rceil]) = \text{src}(\sigma) \} \quad (12)$$

5.2.2 Traces Upper Bound

WCRT of tasks relies on the computation of an upper bound of all the feasible traces of a task with respect to their rbf . According to relation (11), we only need to reason on the maximal feasible traces: two traces arriving at the same state will be extended the same way (12). Therefore we only need to keep the greatest trace among the set of traces that terminate in the same state. We then compute the set

of traces that are maximal using the following recursive equations:

$$\begin{aligned} \mathcal{V}^1 &= \{ \langle \sigma \rangle, \sigma \in T \mid \forall \sigma' \in T, \sigma \neq \sigma' \wedge (tgt(\sigma) \\ &= tgt(\sigma') \implies \langle \sigma \rangle \geq \langle \sigma' \rangle) \} \end{aligned} \quad (13)$$

$$\begin{aligned} \mathcal{V}^{n+1} &= \{ \langle \mathcal{T}, \sigma \rangle, \mathcal{T} \in \mathcal{V}^n, \sigma \in T \mid \forall \mathcal{T}' \in \mathcal{V}^n, \forall \sigma' \in T, \\ &\mathcal{T}' \neq \mathcal{T} \wedge \langle \mathcal{T}, \sigma \rangle \in next(\mathcal{T}) \wedge \langle \mathcal{T}', \sigma' \rangle \in next(\mathcal{T}') \\ &\wedge (tgt(\sigma) = tgt(\sigma') \implies \langle \mathcal{T}, \sigma \rangle \geq \langle \mathcal{T}', \sigma' \rangle) \} \end{aligned} \quad (14)$$

From these trace set, we can extract $\mathcal{T}^+$, an upper bound of all the feasible traces of the task according to Eq. 11 using the following recursive equations:

$$\mathcal{T}_1^+ = \langle \sigma \rangle \mid \sigma = \operatorname{argmax}_{\sigma_i \in \Sigma} \delta(\sigma_i) \quad (15)$$

$$\begin{aligned} \mathcal{T}_{n+1}^+ &= \langle \mathcal{T}_n^+, \mathcal{T}[n+1] \rangle \mid \mathcal{T} \\ &= \operatorname{argmax}_{\mathcal{T}' \in \mathcal{V}^{n+1}} rbf(\mathcal{T}', (n+1) \times T) \end{aligned} \quad (16)$$

The construction of trace $\mathcal{T}^+$ does not ensure that the trace is feasible, i.e. that two consecutive transitions of the trace are effectively consecutive transitions of the PSM. The timing function δ has therefore no meaning on $\mathcal{T}^+$ and the computation of the rbf on $\mathcal{T}^+$ must use a new function δ^+ recursively defined while building $\mathcal{T}^+$ by:

$$\delta^+(\mathcal{T}^+[1]) = \delta(\sigma) \quad (17)$$

$$\delta^+(\mathcal{T}^+[n+1]) = rbf(\mathcal{T}, n+1) - \sum_{j=1}^n \delta^+(\mathcal{T}^+[j]) \quad (18)$$

with σ and $\mathcal{T}$ respectively defined in Eqs. 15 and 16.

5.2.3 WCRT

In order to compute the WCRT, we adapted the usual recursive procedure proposed by [13]. Instead of using the classical task's execution time, we use the upper bound trace's values: at each iteration of the recursive procedure, we use the next iteration of the upper bound trace.

$$\mathcal{R}_i^0 = rbf(\mathcal{T}_i^+, 0) \quad (19)$$

$$\mathcal{R}_i^{n+1} = \sum_{j \in hp(i)} rbf(\mathcal{T}_j^+, \mathcal{R}_i^n) + \mathcal{R}_i^0 \quad (20)$$

In Eq. 20, the sum is made over all traces $\mathcal{T}_j^+$ that have a greater priority than the i -th component (i.e., $j \in hp(i)$). Two conditions stop this recursive loop: (1) whenever two consecutive iterations have the same $\mathcal{R}_i$ value, (2) whenever $\mathcal{R}_i^{n+1}$ reaches the task's deadline D_i .

In the latter the component is not schedulable. Finally, if all the WCRT are lesser than the deadlines (i.e., $\forall i, \mathcal{R}_i \leq D_i$), the complete software architecture is *schedulable*.

6 Code Generation

In order to eventually execute the architecture on the target platform, we have set up a code generation process (see Fig. 7).

It takes as input the MAUVE models of components and architectures specified using the MAUVE DSL (see Section 3). The code generator then translates this model into source files and PSM models of the components (see Section 5.1) in order to apply the analysis process. The source files are completed with the definition of codels, that can either be directly implemented in the generated files, or linked with a third-party library. The resulting software architecture uses the ROS ecosystem for compiling the software, and launch files to help the deployment. The software architecture is only made of Orocos components: Orocos manages the real-time execution and data exchange between components.

The generated sources also embed a tracing library, based on LTTng [65], that allows to produce tracepoints at runtime, including clock values and relevant data about the execution of components (start and end of states and programs, codel calls).

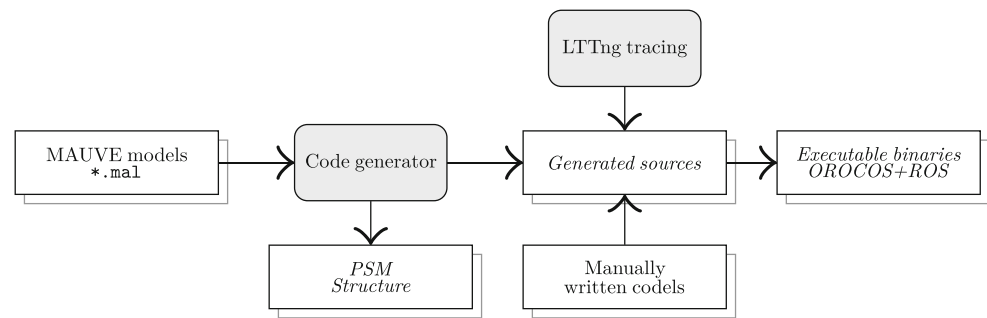
7 Results and Discussion

The methodology presented in this paper has been fully applied to the application presented in Section 2. The deployment specification of the architecture of Fig. 4 is presented in Table 2. The period of the `p3dx_driver` is based on the period of the communication with the low-level processor embedded on the robot. Most of the periods are then set to 100 ms. The navigation and gmapping components are executed with a period of 1 s, as they may be quite time consuming. Moreover, the gmapping component is isolated on the second CPU core. The exploration component is set non real-time: it is executed sporadically when a new exploration point has to be defined.¹

7.1 pWCET Estimations

We have first applied the pWCET estimation method based on measurements to obtain WCETs of component functions. The MBPTA approach (see Section 4.3) uses measures of execution times of the codels, independently of the deployment that is executed. It is then possible to perform independent measurements from several deployments and executions:

¹a non real-time component is not scheduled by the real-time scheduler, and consequently does not disturb the behaviour of real-time components.

Fig. 7 Code generation process

- an *automatic control* deployment, in which we deployed the `p3dx_driver`, `control`, `hokuyo`, `switch` and `guidance` components; we manually sent some guidance goals to the guidance component in a good variability of input conditions such as a densely packed storage area with complex shapes, tight straight corridors and a wide open room with and without moving objects;
- a *mapping* deployment, in which we deployed the necessary components to manually control the robot from user command, while building a map;
- a *navigation* deployment, in which we only deployed the navigation component; the component has been fed with a static global map; we randomly sent several goals to the navigation component without moving the robot.

The exploration component has not been used: it is not executed in real-time, and then is not considered in the real-time analysis. The `p3dx_driver` state machine has only one state (Listing 6). This state executes a set of codelets, for which we have to estimate the WCET. Figure 8a shows some measurements of the execution time of the `aria_loop` codelet, on a small part of the gathered measures.

This codelet is responsible of making the actual communication with the robot controller through a serial connection. Therefore, the communication time with the hardware is taken into account into the measurements. The pWCET estimation of the `aria_loop` codelet is shown in Fig. 8b. From this pWCET estimation, the WCET of the `p3dx_driver` component is estimated according to several possible significance thresholds. Figure 9 shows the PSM for a significance level of 10^{-7} .

7.2 Schedulability Results

From the MAUVE models and the estimated WCETs, we have built the PSM models for all the components and all the desired significance levels.

7.2.1 Evaluating the Impact of pWCET Threshold

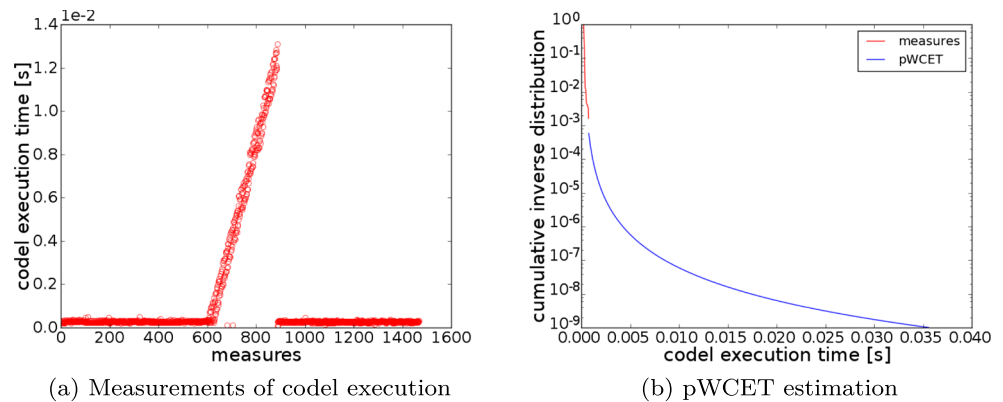
Table 3 shows the resulting WCRT for the several components according to the selected significance level. At 10^{-5} , the navigation component cannot be proved to be schedulable. At 10^{-9} , only the `p3dx_driver` component is schedulable. As the `hokuyo` WCRT overshoots its deadline, it is not possible to compute a WCRT for the less priority components. The `gmapping` component runs on a separate CPU core. The component is schedulable down to a significance of 10^{-7} .

7.2.2 Evaluating the Impact of State-Machines

The method we have presented in this paper takes explicitly state-machines into account when computing the WCRT of components using an upper bound of traces $\mathcal{T}^+$ (let's note this WCRT $\mathcal{R}^+$). We have compared with the computation of the WCRT using the classical method, i.e. by considering that each task has only one WCET. It corresponds to defining for each component a PSM with only one state (like in Fig. 9), whose WCET would be the maximum of the WCET values of the original PSM transitions. Let's note this WCRT $\mathcal{R}^*$. The $\mathcal{R}^*$ for the components running on CPU first core are shown in Table 4.

Table 2 Deployment specification

component	period (ms)	deadline (ms)	priority	CPU
<code>p3dx_driver</code>	100	100	10	1
<code>hokuyo</code>	100	100	9	1
<code>switch</code>	100	100	8	1
<code>pose</code>	100	100	7	1
<code>guidance</code>	100	100	6	1
<code>control</code>	100	100	5	1
<code>teleop</code>	100	100	4	1
<code>navigation</code>	1000	1000	2	1
<code>gmapping</code>	1000	1000	3	2
<code>exploration</code>	—	—	background	—

Fig. 8 MBPTA applied to the `aria_loop` code

Regarding the schedulability of components, the only noticeable situation is for the navigation component at a level of 10^{-4} : it can be proved schedulable using $\mathcal{R}^+$ but not with $\mathcal{R}^*$. The gain obtained with our method can be computed as:

$$\Gamma = \frac{\mathcal{R}^* - \mathcal{R}^+}{\mathcal{R}^*} \quad (21)$$

The value of Γ for the navigation component (i.e. for the whole architecture as the navigation is the less priority component) is 0.077 for a significance level of 10^{-4} (meaning that our method is around 7% less pessimistic), and up to 0.957 for a level of 10^{-8} (meaning that our method is about 95% less pessimistic). The gain is clearly greater when considering components with state machines that have quite different WCET values, like for the navigation component, where the WCET of transitions is of 297 ms when executing the `astar` code, while other transitions have a WCET lower than 3 ms.

7.3 Discussion

The results of Table 3 lead to the following conclusions:

- the whole architecture is schedulable if a significance level of 10^{-4} is acceptable;
- if we need a lower level, down to 10^{-7} , all the architecture is proved to be schedulable, but the navigation component.
- at a significance level of 10^{-8} , all the architecture is proved to be schedulable, but the navigation and `gmapping` components.

**Fig. 9** PSM of the `p3dx_driver` component for a significance level of 10^{-7}

In the other cases, nothing can be deduced regarding the schedulability: we compute an upper bound of the WCRT, and then we cannot conclude that the components are actually schedulable or not.

Besides the direct conclusion regarding schedulability, these results can help designing the architecture. They indeed emphasize some real-time behaviours that may happen during an actual run of the system. Let us consider that we would like to provide a significance level of 10^{-8} . Some possible design choices may be to:

- do nothing: `gmapping` and `navigation` are the less priority components on their respective core, and in the case where they would pass their deadline, they will have no impact on the other components;
- change the affinity of the navigation component: by moving it to a core where it will be the only real-time component, it may be proved to be schedulable, which may be mandatory if its functions are critical for the safety of the system;

Table 3 WCRT results for all components running on the CPU. Values are in milliseconds. Values in bold are exceeding the component deadline

Core 1 component	1e-3	1e-4	1e-5	1e-6	1e-7	1e-8	1e-9
<code>p3dx_driver</code>	8	10	11	13	17	29	55
<code>hokuyo</code>	48	53	55	58	63	75	156
<code>switch</code>	51	56	58	63	69	84	—
<code>pose</code>	52	57	59	64	70	85	—
<code>guidance</code>	55	60	62	67	73	88	—
<code>control</code>	58	63	65	70	76	91	—
<code>teleop</code>	65	70	72	77	83	98	—
<code>navigation</code>	882	997	1017	1067	1127	1277	—
Core 2 component	1e-3	1e-4	1e-5	1e-6	1e-7	1e-8	1e-9
<code>gmapping</code>	789	789	791	798	839	1069	2333

Table 4 $\mathcal{R}^*$ for components running on core 1. Values are in milliseconds. Values in bold are exceeding the component deadline

component	1e-3	1e-4	1e-5	1e-6	1e-7	1e-8	1e-9
p3dx_driver	8	10	11	13	17	29	55
hokuyo	48	53	55	58	63	75	156
switch	51	57	59	64	70	85	–
pose	52	58	60	65	71	86	–
guidance	55	61	63	68	74	89	–
control	58	64	66	71	77	92	–
teleop	65	71	73	78	84	99	–
navigation	884	1080	1175	1391	1895	29900	–

- let some components run in *background* mode: they will not be real-time any more, meaning that they may pass their deadline, but also that they may run faster than their period when the CPU is under loaded; it may lead to more efficient applications but can be done only for non safety critical components.

In the context of safety critical applications, it is mandatory to perform such real-time analyses to ensure that the system cannot produce unsafe behaviours. More importantly our work allows the designer to apprehend the real-time behaviour of its application and take it into account in the design phase. In the case of the P3-DX robot, violating real-time constraints may for instance lead to decreasing the localization quality (in case the map computed by the gmapping component is distorted), or to possible collisions in case the guidance or switch components are delayed.

8 Conclusion

Safety critical applications demand for sound development processes and for analyses that help proving that the system cannot produce unsafe behaviours. Such analyses must encompass both the functional properties, i.e., verifying that the algorithms provide the good results, and the non-functional properties, among which verifying that the algorithms execute on time. In this paper, we have focused on verifying such real-time constraints as they are necessary to guarantee a correct and safe behavior of the whole system.

In this paper, we have presented a complete toolchain for the design and analysis of component-based real-time software architectures. This toolchain settles on the MAUVE DSL to model components, architectures and deployments. The toolchain comes with code generation towards Orocos real-time tasks. The generated code moreover embeds trace-points to provide timed traces of execution runs. The

real-time analysis takes benefit of the behaviour of components to compute a WCRT less pessimistic than classical approaches that consider components as a unique function.

We have illustrated the whole process on the design of a software architecture for a ground robot performing an exploration mission. We have first shown the architecture design aspects, with code snapshots of the MAUVE models of components and architectures. We have then discussed and illustrated how timed traces can be used to estimate the WCET of the elementary functions called by each component. Finally, we have run the complete real-time analysis process, and discussed how the results can lead either to prove the real-time safety of critical parts of the architecture, and help the designer make some conception choices about its deployment.

Future works are twofold. Regarding the real-time analysis, it would be interesting to avoid selecting a significance level before computing the WCRT, and we are working on how to directly propagate the pWCET continuous law inside the WCRT computation in order to compute a pWCRT. It would lead to even less pessimistic results as some probabilistic dependency between components and/or states could be taken into account.

Secondly, the WCRT computation uses an execution model that is over pessimistic because we suppose that every task clock are not synchronized; removing this assumption would lead to less complexity in the computation of traces upper bounds and WCRT; This behaviour is due to some limitations of the Orocos middleware regarding real-time synchronisation, and we are considering to make the middleware evolve to fix this behaviour [66].

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

References

1. Freitas, G., Hamner, B., Bergerman, M., Singh, S.: A practical obstacle detection system for autonomous orchard vehicles. In: International Conference on Intelligent Robots and Systems (IROS), Vilamoura-Algarve, Portugal (2012)
2. Haddadin, S., Haddadin, S., Khoury, A., Rokahr, T., Parusel, S., Burgkart, R., Bicchi, A., Albu-Schaffer, A.: A truly safely moving robot has to know what injury it may cause. In: International Conference on Intelligent Robots and Systems (IROS), Vilamoura-Algarve, Portugal (2012)
3. Lens, T., Von Stryk, O.: Investigation of safety in human-robot-interaction for a series elastic, tendon-driven robot arm. In: International Conference on Intelligent Robots and Systems (IROS), Vilamoura-Algarve, Portugal (2012)
4. Rybski, P., Anderson-Sprecher, P., Huber, D., Niessl, C., Simmons, R.: Sensor fusion for human safety in industrial workcells. In: International Conference on Intelligent Robots and Systems (IROS), Vilamoura-Algarve, Portugal (2012)
5. Tamura, Y., Dai Le, P., Hitomi, K., Chandrasiri, N., Bando, T., Yamashita, A., Asama, H.: Development of pedestrian behavior

- model taking account of intention. In: International Conference on Intelligent Robots and Systems (IROS), Vilamoura-Algarve, Portugal (2012)
6. Elbaum, S., Detweiler, C.: Reducing failure rates of robotic systems through inferred invariants monitoring. In: International Conference on Intelligent Robots and Systems (IROS), Tokyo, Japan (2013)
 7. Nakamura, A., Nagata, K., Harada, K., Yamanobe, N., Tsuji, T., Foissotte, T., Kawai, Y.: Error recovery using task stratification and error classification for manipulation robots in various fields. In: International Conference on Intelligent Robots and Systems (IROS), Tokyo, Japan (2013)
 8. Decastro, J., Kress-Gazit, H.: Guaranteeing reactive high-level behaviors for robots with complex dynamics. In: International Conference on Intelligent Robots and Systems (IROS), Tokyo, Japan (2013)
 9. Pathak, S., Pulina, L., Metta, G., Tacchella, A.: Ensuring safety of policies learned by reinforcement reaching objects in the presence of obstacles with the iCub. In: International Conference on Intelligent Robots and Systems (IROS), Tokyo, Japan (2013)
 10. Cervin, A., Lincoln, B., Eker, J., Årzén, K.-E., Buttazzo, G.: The Jitter Margin and its application in the design of real-time control systems. In: International Conference on Real-Time and Embedded Computing Systems and Applications (RTCSA), Göteborg, Sweden (2004)
 11. Henriksson, D., Cervin, A., Åkesson, J., Årzén, K.-E.: On dynamic real-time scheduling of model predictive controllers. In: International Conference on Decision and Control (CDC), Las Vegas, NV, USA (2002)
 12. Klein, M.H., Ralya, T., Pollak, B., Obenza, R., Harbour, M.G.: A practitioner's handbook for real-time analysis. Kluwer Academic Publishers, Dordrecht (1993)
 13. Liu, L., James, C., Layland, W.: Scheduling algorithms for multiprogramming in a hard-real-time environment scheduling algorithms for multiprogramming. *J. Assoc. Comput. Mach.* **20**(1), 46–61 (1973)
 14. Spuri, M.: Analysis of deadline scheduled real-time systems technical report (1996)
 15. Sha, L., Rajkumar, R., Lehoczky, J.P.: Priority inheritance protocols: an approach to real-time synchronization. *IEEE Trans. Comput.* **39**(9), 1175–1185 (1990)
 16. Chen, M.-I., Lin, K.-J.: Dynamic priority ceilings: a concurrency control protocol for real-time systems. *Real-Time Syst.* **2**(4), 325–346 (1990)
 17. Baker, T.P.: Stack-based scheduling of realtime processes. *Real-Time Syst.* **3**(1), 67–99 (1991)
 18. Chetto, H., Silly, M., Bouchentouf, T.: Dynamic scheduling of real-time tasks under precedence constraints. *Real-Time Syst.* **2**(3), 181–194 (1990)
 19. collab=Erann Gat: On three-layer architectures. In: Artificial Intelligence and Mobile Robots (1998)
 20. Albus, J.S.: 4D/RCS: a reference model architecture for intelligent unmanned ground vehicles. *AeroSense 2002* (2002)
 21. McGann, C., Py, F., Rajan, K., Thomas, H., Henthorn, R., McEwen, R.: A Deliberative Architecture for AUV Control. In: International Conference on Robotics and Automation (ICRA), Pasadena, California, USA (2008)
 22. Muscettola, N., Dorais, G., Fry, C., Levinson, R., Plaunt, C.: IDEA planning at the core of autonomous reactive agents. In: International NASA Workshop on Planning and Scheduling for Space (IWPSS), Houston, TX, USA (2002)
 23. Volpe, R., Richard, I., Nesnas, I., Estlin, T., Tara, D., Mutz, D., Petras, R., Das, H.: The CLARATy Architecture for Robotic Autonomy. In: IEEE Aerospace Conference, Big Sky, MT, USA (2001)
 24. Brugali, D., Scandurra, P.: Component-based robotic engineering (Part I). *IEEE Robot. Autom. Mag.* **16**(4), 84–96 (2009)
 25. Brugali, D., Shakhimardanov, A.: Component-based robotic engineering (Part II). *IEEE Robot. Autom. Mag.* **17**(1), 100–112 (2010)
 26. Jung, M.Y., Balicki, M., Taylor, R.H., Kazanzides, P.: Lessons learned from the development of component-based medical robot systems. *J. Softw. Eng. Robotics* **5**(2), 25–41 (2013)
 27. Ando, N., Kurihara, S., Biggs, G., Sakamoto, T., Nakamoto, H., Kotoku, T.: Software deployment infrastructure for component based rt-systems. *J. Rob. Mechatronics* **23**(3), 350–359 (2011)
 28. Manso, L., Bachiller, P., Bustos, P., Nunez, P., Cintas, R., Calderita, L.: RoboComp: a tool-based robotics framework. In: International Conference on Simulation, Modelling and Programming for Autonomous Robots (SIMPAN), Darmstadt, Germany (2010)
 29. Martínez, J., Romero-Garcés, A., Manso, L., Bustos, P.: Improving a robotics framework with real-time and high-performance features. In: International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAN), Darmstadt, Germany (2010)
 30. Soetens, P., Bruyninckx, H.: Realtime hybrid task-based control for robots and machine tools. In: International Conference on Robotics and Automation (ICRA), Barcelona, Spain (2005)
 31. Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Berger, E., Wheeler, R., Mga, A.: ROS: an open-source robot operating system. In: ICRA Workshop on Open Source Software, Kobe, Japan (2009)
 32. Basu, A., Gallien, M., Lesire, C., Nguyen, T.-h., Bensalem, S., Ingrand, F., Sifakis, J.: Incremental component-based construction and verification of a robotic system. In: European Conference on Artificial Intelligence (ECAI), Patras, Greece (2008)
 33. Navet, N., Loïc, F.: CPAL high-level abstractions for safe embedded systems. In: Proceedings of the International Workshop on Domain-specific Modeling, DSM 2016, pp. 35–41, New York, NY, USA. ACM (2016)
 34. Rock, the robot construction kit. <http://www.rock-robotics.org/stable/>
 35. Bruyninckx, H., Klotzbücher, M., Hochgeschwender, N., Kraetzschmar, G., Gherardi, L., Brugali, D.: The BRICS component model: a model-based development paradigm for complex robotics software systems. In: ACM Symposium on Applied Computing (SAC), Coimbra, Portugal (2013)
 36. Mallet, A., Pasteur, C., Herrb, M.: GenoM3: building middleware-independent robotic components. In: International Conference on Robotics and Automation (ICRA), Anchorage, AK, USA (2010)
 37. Foughali, M., Berthomieu, B., Dal Zilio, S., Ingrand, F., Mallet, A.: Model checking real-time properties on the functional layer of autonomous robots. In: International Conference on Formal Engineering Methods (ICFEM), Tokyo, Japan (2016)
 38. Schlegel, C., Steck, A., Brugali, D., Knoll, A.: Design abstraction and processes in robotics: from code-driven to model-driven engineering. In: International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAN), Darmstadt, Germany (2010)
 39. Steck, A., Schlegel, C.: Towards quality of service and resource aware robotic systems through model-driven software development. In: International Workshop on Domain-Specific Languages and Models for Robotic Systems (DSLRob), Taipei, Taiwan (2010)
 40. Singhoff, F., Legrand, J., Nana, L., Marcé, L.: Cheddar: a flexible real time scheduling framework. *ACM SIGAda Ada Lett.* **24**(4), 1–8 (2004)
 41. Gobillot, N., Lesire, C., Dooze, D.: A modeling framework for software architecture specification and validation. In: International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAN), Bergamo, Italy (2014)

42. Gobillot, N., Doose, D., Lesire, C., Santinelli, L.: Periodic state-machine aware real-time analysis. In: *Emerging Technologies and Factory Automation (ETFA)*, Luxembourg, Luxembourg (2015)
43. Gobillot, N., Guet, F., Doose, D., Grand, C., Lesire, C., Santinelli, L.: Measurement-based real-time analysis of robotic software architectures. In: *International Conference on Intelligent Robots and Systems (IROS)*, Daejeon, South Korea (2016)
44. Watanabe, Y., Manecy, A., Amiez, A., Lesire, C., Grand, C.: Non-cooperative ground vehicle tracking and interception by multi-RPA collaboration. In: *International Council of the Aeronautical Sciences (ICAS)*, Daejeon, South Korea (2016)
45. Feiler, P., Gluch, D.: *Model-based engineering with AADL: an introduction to the SAE architecture analysis & design language*. Addison-Wesley Professional, Boston (2012)
46. Varró, D.: A formal semantics of UML statecharts by model transition systems. In: *International Conference on Graph Transformation (ICGT)*, Barcelona, Spain (2002)
47. Selic, B., Gérard, S.: *Modeling and analysis of real-time and embedded systems with UML and MARTE: developing Cyber-Physical systems*. Elsevier, Amsterdam (2013)
48. Biggs, G., Fujiwara, K., Anada, K.: Modelling and analysis of a redundant mobile robot architecture using AADL. In: *International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAN)*, Bergamo, Italy (2014)
49. Bardaro, G., Semperebon, A., Matteucci, M.: AADL For robotics: a general approach for system architecture modeling and code generation. *J. Softw. Eng. Robotics (JOSER)* **8**(1), 32–44 (2017)
50. Hart, P., Nilsson, N., Raphael, B.: A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Syst. Sci. Cybern.* **4**(2), 100–107 (1968)
51. Szyperski, C., Gruntz, D., Stephan, M.: *Component software: beyond object-oriented programming*. Addison-Wesley, Reading (2002)
52. Schlegel, C.: Communication patterns as key towards component-based robotics. *J. Adv. Robotic Syst.* **3**(1), 49–54 (2006)
53. Klotzbücher, M., Bruyninckx, H.: Coordinating robotic tasks and systems with rFSM statecharts. *J. Softw. Eng. Robotics (JOSER)* **1**(1), 28–56 (2012)
54. Gobillot, N., Lesire, C., Doose, D.: A component-based navigation-guidance-control design pattern for mobile robots. In: *ICRA Workshop on Software Development and Integration for Robotics (SDIR)*, Karlsruhe, Germany (2013)
55. Herlihy, M.: Wait-free synchronization. *ACM Trans. Program. Lang. Syst.* **13**(1), 124–149 (1991)
56. Wilhelm, R., Mitra, T., Mueller, F., Puaut, I., Puschner, P., Staschulat, J., Stenström, P., Engblom, J., Ermedahl, A., Holsti, N., Thesing, S., Whalley, D., Bernat, G., Ferdinand, C., Heckmann, R.: The worst-case execution-time problem-overview of methods and survey of tools. *ACM Trans. Embedded Comput. Syst.* **7**(3), 36:1–36:53 (2008)
57. Ferdinand, C.: Worst case execution time prediction by static program analysis. In: *International Parallel and Distributed Processing Symposium (IPDPS)*, Santa Fe, NM, USA (2004)
58. Hansen, Jeffery., Hissam, S.cott., Moreno, Gabriel.: *Statistical-Based WCET Estimation and Validation*. Dublin, Ireland
59. Cucu-Grosjean, L., Santinelli, L., Houston, M., Lo, C., Vardanega, T., Kosmidis, L., Abella, J., Mezzetti, E., Quiñones, E., Cazorla, F.: Measurement-based probabilistic timing analysis for multi-path programs. In: *Euromicro Conference on Real-Time Systems (ECRTS)*, Pisa, Italy (2012)
60. Mok, A., Chen, D.: A multiframe model for real-time tasks. *IEEE Trans. Softw. Eng.* **23**(10), 635–645 (1997)
61. Moyo, N.T., Nicollet, E., Lafaye, F., Moy, C.: On schedulability analysis of non-cyclic generalized multiframe tasks. In: *Euromicro Conference on Real-Time Systems (ECRTS)*, Brussels, Belgium (2010)
62. Stigge, M., Ekberg, P., Guan, N., Yi, W.: The digraph real-time task model. In: *Real-Time and Embedded Technology and Applications Symposium (RTAS)*, Chicago, IL, USA (2011)
63. Zeng, H., Di Natale, M.: Schedulability analysis of periodic tasks implementing synchronous finite state machines. In: *2012 24th Euromicro Conference on Real-Time Systems* (2012)
64. Baruah, S., Rosier, L.E., Howell, R.R.: Algorithms and complexity concerning the preemptive scheduling of periodic, real-time tasks on one processor. *Real-Time Syst.* **2**, 301–324 (1990)
65. Desnoyers, M., Dagenais, M.: The LTTng tracer: a low impact performance and behavior monitoring for GNU/Linux. In: *Linux Symposium*, Ottawa, Canada (2006)
66. Doose, D., Grand, C., Lesire, C.: MAUVE runtime: a component-based middleware to reconfigure software architectures in real-time. *J. Softw. Eng. Robotics (JOSER)* **8**(1), 128–140 (2017)

Nicolas Gobillot received his M.Eng. degree in Electronics, Robotics and Computer Sciences from Polytech' Montpellier in 2012 and his Ph.D. degree in Computer Sciences from University of Toulouse, France in 2016. From 2016 to 2018, he was a post-doctoral fellow at CEA in the robotics department. Since 2018 he has been a research engineer at Ifremer, in the Localisation, Robotics, Acoustics and Optics (PRAO) department. His main topics of interest include real-time systems, embedded systems and software engineering for robotics.

Charles Lesire received his M.Sc. degree in Computer Sciences from ENAC, Toulouse, France in 2003, and the Ph.D. degree in Computer Sciences from University of Toulouse, France, in 2006. In 2007, he was a post-doctoral fellow at LAAS-CNRS, in the robotics team. From 2008 to 2016, he was a research fellow at ONERA in the System Control department (DCSD). Since 2017, he has been a research fellow at ONERA in the Information Processing department (DTIS), and is now head of the Safe, Embedded and Autonomous Systems lab at ONERA. His main topics of interest include software engineering for robotics, embedded decision-making, and cooperation of heterogeneous robots. He is a member of the IEEE RAS Technical Committee on Software Engineering for Robotics and Automation (TC-SOFT).

David Doose received his M.Sc. degree in Computer Sciences from University Paul Sabatier, Toulouse, France in 2002, and the Ph.D. degree in Computer Sciences from University of Toulouse, France, in 2006. In 2007, he was a post-doctoral fellow at University of Toulouse in collaboration with Airbus group. Since 2007, he has been a research fellow at ONERA, in the Computer Science department. His main topics of interest include embedded systems, real-time analysis, formal methods and recently software engineering for robotics.